

RELATÓRIO TÉCNICO - PARTE TEÓRICA

Modernização da Plataforma de Pedidos Loja Veloz: Do Dev ao Prod

Curso: Engenharia Cloud DevOps

Aluno: João Pereira

Papel: Cloud DevOps Engineer Sênior

1. CONCEITUAÇÃO DE ARQUITETURA DE MICROSERVIÇOS E O PAPEL DO DEVOPS

1.1 Arquitetura de Microsserviços

A arquitetura de microsserviços consiste em uma abordagem de design de software na qual uma aplicação complexa é decomposta em um conjunto de serviços autônomos, fracamente acoplados e altamente coesos. Cada serviço é responsável por um domínio de negócio bem definido, executa seu próprio processo de forma independente e se comunica com outros serviços através de protocolos leves, tipicamente APIs HTTP/REST ou mensageria assíncrona.

Ao contrário do modelo monolítico — onde toda a lógica reside em uma única unidade implantável —, os microsserviços oferecem vantagens significativas para o crescimento acelerado de empresas como a Loja Veloz:

- Escalabilidade Granular:** serviços com maior demanda (como criação de pedidos e verificação de estoque) podem ser escalados de forma independente, sem duplicar toda a aplicação.
- Agilidade e Autonomia de Equipes:** times independentes podem programar, testar e implantar seus microsserviços sem depender do ciclo de release global de outros times.

- **Poliglotismo Tecnológico:** cada serviço pode ser escrito na linguagem e banco de dados que melhor se adaptem ao seu escopo (ex.: Node.js para requisições rápidas de API, PostgreSQL para transações ACID).
- **Isolamento de Falhas:** uma pane crítica em um serviço (instabilidade no gateway de pagamentos) não paralisa o sistema de listagem de produtos.

1.2 O Papel do DevOps em Ambientes Cloud-Native

O movimento DevOps une metodologias de Desenvolvimento e Operações sob uma cultura de colaboração, automação e feedback rápido. Em ambientes Cloud-Native, onde recursos de computação são abstratos e efêmeros, gerenciar sistemas manualmente torna-se um erro operacional grave.

O DevOps atua introduzindo:

1. **Infraestrutura como Código (IaC):** definição da infraestrutura física por meio de arquivos declarativos, garantindo reprodutibilidade e eliminação de alterações manuais destrutivas.
 2. **Automação Extrema (CI/CD):** eliminação de lançamentos manuais propensos a falhas. Cada alteração de código passa por testes e validações automáticas antes de ir para produção.
 3. **Observabilidade Avançada:** ferramentas de telemetria distribuída para que os times possam diagnosticar falhas de comunicação em tempo real.
 4. **Engenharia de Confiabilidade (SRE):** garantia de que a aplicação se autogestione na nuvem, reagindo automaticamente a falhas e picos.
-

2. CONTEINERIZAÇÃO: DIFERENÇAS ENTRE DOCKER E KUBERNETES

2.1 O Conceito de Containerização

A containerização é uma tecnologia de virtualização a nível de sistema operacional que encapsula o código de uma aplicação, suas dependências, binários e bibliotecas de runtime em uma única unidade lógica chamada **Container**.

Diferente das máquinas virtuais tradicionais — que exigem um hipervisor e um sistema operacional completo para cada instância, consumindo gigabytes de memória —, os contêineres são isolados por primitivas do Kernel Linux (Namespaces e Cgroups), iniciando em frações de segundo e consumindo apenas os recursos de que a aplicação necessita.

2.2 Diferenças Críticas: Docker vs. Kubernetes (K8s)

Critério	Docker	Kubernetes
Escopo	Container individual. Compila e roda localmente	Orquestração de centenas distribuídos em cluster
Ambiente Alvo	Desenvolvimento local e testes unitários	Produção elástica e alta disponibilidade
Abordagem	Imperativa (<code>docker build</code> , <code>docker run</code>)	Declarativa (manifestos YAML com estado desejado)
Resiliência	Reinicialização básica no mesmo host	Recria containers em servidores saudáveis
Balanceamento	Estático, via portas locais	Dinâmico via Services, Ingress e Service Mesh

2.3 Quando Utilizar Cada Abordagem

Docker (e Docker Compose)

Deve ser utilizado durante o ciclo diário de desenvolvimento. Permite que um desenvolvedor suba toda a stack — API Gateway, microsserviços e banco de dados — localmente com um único comando, isolando dependências sem poluir o sistema operacional local.

Kubernetes

Deve ser utilizado para hospedar a aplicação em produção. À medida que a Loja Veloz cresce, gerenciar contêineres isolados em produção geraria sobrecarga. O Kubernetes gerencia falhas de servidores automaticamente, distribui a carga de rede entre múltiplos nós, escala pods dinamicamente e garante atualizações sem interrupções.

3. FUNDAMENTAÇÃO TEÓRICA

3.1 Orquestração de Containers

Orquestrar contêineres significa gerenciar de forma automatizada o ciclo de vida, agendamento, escalabilidade, rede e armazenamento de centenas de contêineres espalhados em um cluster.

O Kubernetes opera em ciclo contínuo de controle:

1. **Monitorar:** lê o estado atual do cluster.
2. **Comparar:** confronta o estado atual com o estado desejado nos YAMLS.
3. **Agir:** toma ações corretivas (recriar container que falhou no healthcheck) para forçar a convergência ao estado desejado.

3.2 CI/CD em Ambientes Distribuídos

Em sistemas de microserviços, a Integração Contínua (CI) e a Entrega Contínua (CD) são vitais para evitar o caos.

Integração Contínua (CI)

Garante que cada Pull Request seja integrado ao tronco principal de forma segura. O pipeline compila a imagem Docker, executa lints estáticos e roda suítes de testes unitários.

Entrega Contínua (CD)

Automatiza o empacotamento da imagem no Container Registry com versionamento por hash único do Git. Em seguida, injeta a nova imagem nos manifestos do Kubernetes, disparando estratégias seguras de implantação para minimizar riscos operacionais.

3.3 Conceitos de Observabilidade

Diferente do monitoramento tradicional (sinal binário: servidor ativo ou inativo), a observabilidade permite entender o estado interno de um sistema distribuído a partir dos **Três Pilares**:

1. Métricas

Dados agregados e mensuráveis ao longo do tempo (uso de CPU, latência média da API, taxa de erro HTTP por segundo). Úteis para detectar anomalias e acionar alertas.

2. Logs

Registros de texto de eventos específicos com timestamps exatos. Essenciais para depuração aprofundada pós-incidente.

3. Traces

Rastreamento ponta a ponta do caminho percorrido por uma requisição HTTP através de múltiplos microsserviços. Permite localizar exatamente qual serviço está causando latência ou erro em requisições distribuídas.

4. JUSTIFICATIVA DAS DECISÕES ARQUITETURAIS

1. Metodologia 12-Factor App

Força o desacoplamento de configurações e serviços de apoio. As credenciais da Loja Veloz são mantidas fora do código-fonte, injetadas como variáveis de ambiente via Kubernetes Secrets, promovendo portabilidade completa entre ambientes.

2. Dockerfiles Multi-stage e Segurança Non-Root

Imagens multi-stage mantêm ferramentas de build separadas do runtime final. A imagem final usa Alpine com o usuário `node` sem privilégios (`USER node`), diminuindo o tamanho das imagens (FinOps) e bloqueando ataques de escape de container.

3. Kubernetes Probes

Definição obrigatória de `livenessProbe` e `readinessProbe` nos endpoints `/healthz` e `/ready` . Se um container subir com bug catastrófico, ele falhará no teste e o Kubernetes pausará o deploy, mantendo o tráfego nos pods antigos — garantindo zero downtime.

4. Horizontal Pod Autoscaler

A Loja Veloz operará de forma elástica. O HPA baseado em CPU (70%) e Memória (80%) garante provisionamento automático de réplicas em picos abruptos de tráfego, encolhendo recursos em períodos ociosos para otimizar custos.

5. Rolling Update vs Blue/Green

Adotamos Rolling Update nativo no Kubernetes. Garante alta disponibilidade sem exigir duplicação imediata dos custos de infraestrutura que o Blue/Green completo exigiria.

6. Terraform para IaC

O uso do Terraform como ferramenta declarativa impede o *configuration drift* e documenta a infraestrutura da nuvem de forma reproduzível em repositórios Git, facilitando a governança corporativa.

FIM DO RELATÓRIO TEÓRICO